

Multimodal Systems

Maurizio Meschi & Nicolò Trebino
5194399 & 5182167

December 10, 2025

maurizio.meschi@gmail.com,
nicolo.trebino@gmail.com

Contents

- 1 Introduction** **2**
- 2 Sensor Devices for Motion Analysis** **2**
- 3 System Design** **3**
 - 3.1 Architecture Overview 3
 - 3.2 MediaPipe vs YOLO: Implementation Choices 4
- 4 Critical Implementation Aspects** **5**
 - 4.1 Landmark Stability and Noise 5
 - 4.2 World Coordinate Representation 5
- 5 Python Code** **6**

1 Introduction

The goal of this project is to design and implement a multimodal system capable of analyzing human movement based on 3D trajectories of body landmarks. The system computes the kinetic energy of movement for different body subsets (whole body, upper body, lower body, right arm, left arm, right leg, left leg) and visualizes how the total kinetic energy is distributed among them. Additionally, the system detects whether one subset moves significantly more than the others, providing an intuitive textual output.

To meet these objectives, we developed a complete [Python prototype](#) implementing two parallel approaches:

1. **MediaPipe Pose**, which is fast and suited for single-person tracking,
2. **YOLO-based Pose Estimation**, which supports multi-person analysis but requires more computational power.

The following sections illustrate the sensors that can be employed for such a system, the architecture we designed, critical implementation aspects, and the developed Python prototype.

2 Sensor Devices for Motion Analysis

To support 3D landmark extraction and motion analysis, several categories of sensing devices can be adopted. Table 1 summarizes the main options together with their advantages and limitations.

Sensor Type	Advantages	Limitations
RGB Cameras*	• Low cost, widely available • Easy to integrate and deploy • Do not require markers	• Only pseudo-3D (depth inferred) • Sensitive to lighting • Occlusions reduce accuracy
Depth Cameras	• Provide real depth information • More accurate 3D joint estimation • Robust to lighting changes	• Higher cost • Limited performance outdoors (IR interference)
IMUs (Wearable)	• Direct measurement of acceleration and rotation • Not affected by lighting or occlusions • Good for sport or rehabilitation analysis	• Require wearing sensors on the body • Suffer from drift and calibration issues • Multi-sensor setups require synchronization
Optical Motion Capture Systems	• Extremely high accuracy • Provide true 3D measurements	• Very expensive • Require controlled environments and markers

Table 1: Comparison of different sensor devices for motion analysis.

*Approach used in this project.

3 System Design

3.1 Architecture Overview

The overall architecture of the system can be interpreted following the hierarchical structure shown in Figure 1, where each processing level transforms the input signal into increasingly abstract representations. In our design, every stage of the stack corresponds to a specific module of the implemented system.

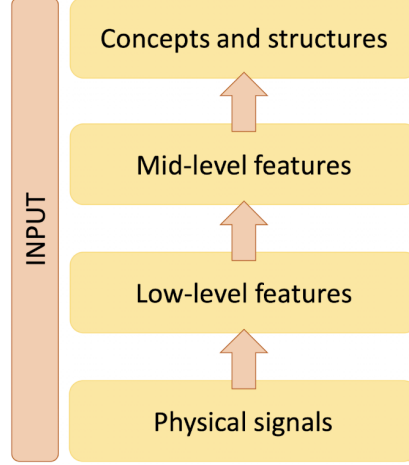


Figure 1: Layered Framework - from physical signals to high-level concepts.

Physical Signals: Pose Acquisition

At the bottom of the stack lie the **physical signals**, which correspond in our case to the raw visual input stream. The system supports two types of acquisition:

- **offline input**: videos from a motion dataset,
- **live input**: webcam stream processed in real time.

These signals are processed by the **Pose Estimation Module**, implemented using MediaPipe Pose and a YOLO-based pose model.

At this level, the system extracts a set of n body landmarks for each detected person:

- **Mediapipe** ($n = 33$)

$$P(t_i) = \{\mathbf{p}_1(t_i), \mathbf{p}_2(t_i), \dots, \mathbf{p}_n(t_i)\}, \quad \mathbf{p}_k = [x_k, y_k, z_k] \in \mathbb{R}^3$$

- **YOLO** ($n = 17$)

$$P(t) = \{\mathbf{p}_1(t_i), \mathbf{p}_2(t_i), \dots, \mathbf{p}_n(t_i)\}, \quad \mathbf{p}_k = [x_k, y_k, c_k] \in \mathbb{R}^3$$

where c_k is the confidence assigned to the estimation k .

This representation constitutes the bridge from raw sensor data to symbolic body-structure information.

Low-level Features: Velocities and Kinetic Energies

The next level of the stack contains **low-level features**, which in our system are derived from temporal variations in the landmark positions.

Given the positions $P(t)$, the velocity of each joint is computed as the first-order temporal derivative:

$$\mathbf{v}_k(t_i) = \frac{\mathbf{p}_k(t_i) - \mathbf{p}_k(t_{i-1})}{t_i - t_{i-1}}$$

Using an anthropometric table and the total body mass of the subject, we assign a mass value to each joint. For each anatomical group (right arm, left arm, right leg, left leg, upper body, lower body), we compute the group kinetic energy as:

$$KE_{\text{group}}(t_i) = \frac{1}{2} \sum_{j \in \text{group}} m_j \|\mathbf{v}_j(t_i)\|^2$$

Mid-level Features: Unitizing by Thresholding

The **mid-level features** represent structural patterns derived from the low-level kinetic energy values. At this stage, we aim to detect which limb is moving more prominently than the others. We apply a **unitizing-by-thresholding** process: a limb is considered *dominant* if its kinetic energy exceeds twice the energy of all other limbs:

$$KE_{\text{dominant}}(t_i) > 2 \cdot KE_{\text{others}}(t_i)$$

This rule provides a clear and robust symbolic interpretation of movement intensity. This stage abstracts movement from raw geometry to discrete symbolic events.

High-level Interpretation

Then the system generates understandable, human-level descriptions of movement behavior. The bar chart of energy distribution and the textual message form the final, interpretable output. This level integrates all previous representations into meaningful feedback that can be used in gesture analysis, rehabilitation, sports training, or human-computer interaction.

3.2 MediaPipe vs YOLO: Implementation Choices

MediaPipe Pose

- provides fast, lightweight pose tracking,
- ideal for single-person interactions,
- easy integration and good stability.

YOLO-based Pose Estimation

- supports multi-person scenarios,
- higher flexibility,
- better suited for videos with complex movement.

This dual approach allows adapting the system to different environments: low-resource real-time applications can use MediaPipe, while high-precision multi-person tracking benefits from YOLO.

4 Critical Implementation Aspects

Developing such a system required addressing several challenges:

4.1 Landmark Stability and Noise

A critical aspect in the computation of joint velocities and kinetic energy is the stability of the landmark trajectories. Both MediaPipe and YOLO-based pose estimation introduce frame-to-frame jitter due to model uncertainty, partial occlusions, and image noise. Since velocity is computed as a temporal derivative of position, even small fluctuations in landmark coordinates can produce disproportionately large and unrealistic velocity spikes, which would in turn corrupt the kinetic energy estimation.

To mitigate this issue, we applied a multichannel **Butterworth low-pass filter** to the landmark *positions* before velocity computation. Filtering the positions instead of the velocities is the correct approach for two reasons:

1. **Derivatives amplify high-frequency noise.** If velocities were filtered directly, the noise amplification would have already occurred during differentiation, making it harder to recover smooth and physically plausible trajectories. By smoothing the positions first, the derivative naturally becomes stable, resulting in a more accurate estimate of instantaneous joint velocity.
2. **Kinetic energy is quadratic in velocity.** Since $KE = \frac{1}{2}m\|v\|^2$, even minimal noise in the velocity signal would be squared, dramatically affecting the final kinetic energy. Ensuring smooth input trajectories prevents these artefacts at their source.

Mediapipe: the filter operates jointly on all landmark coordinates, ensuring temporal coherence across the entire skeleton. Each new frame is passed through the low-pass filter before computing the velocity difference between consecutive frames.

YOLO: an additional challenge arises, in fact each detected person must be filtered independently. Since the Butterworth filter maintains an internal temporal state, applying a single global filter would blend the trajectories of different individuals, corrupting both velocity and kinetic energy computation. To address this, the system maintains a separate filter instance for each tracked person ID, ensuring that the temporal smoothing is consistent for every individual across frames.

This filtering strategy significantly improves the stability of computed velocities, prevents unrealistic kinetic energy spikes, and ensures that the subsequent mid-level analysis (such as dominant limb detection) is based on physically meaningful motion signals.

4.2 World Coordinate Representation

To improve the realism and comparability of our results, we convert all detected keypoints into an approximate **world coordinate system**.

MediaPipe: Built-in World Landmarks

MediaPipe provides a direct mapping from 2D image landmarks to 3D *world landmarks*, expressed in a metric-like, camera-normalized reference frame. Using these world coordinates greatly improves the stability and physical interpretability of the computed velocities and consequently of the kinetic energy.

YOLO: Heuristic World Coordinate Approximation

Unlike MediaPipe, YOLO-based pose models only provide 2D pixel coordinates and a confidence score. To obtain physically meaningful positions, we implemented a widely used anthropometric heuristic that reconstructs an approximate world-scale representation of the skeleton.

The conversion relies on the following assumptions:

- the real height of the subject is known or approximated (e.g., average human height),
- certain upper-body and lower-body keypoints (nose, eyes, shoulders, ankles) can be used to estimate the pixel height of the person in the image,
- the ratio between real body dimensions (e.g., shoulders width, height proportions) and their representation in pixels provides a scale factor.

This scale factor is then used to convert every pixel coordinate into meters. The result is an approximate but physically consistent 3D coordinate frame in which velocity and kinetic energy become meaningful quantities.

5 Python Code

The implemented prototype provides:

- video input (live or from dataset),
- MediaPipe or YOLO-based pose estimation,
- kinetic energy computation for each subset,
- energy distribution visualization,
- automatic textual analysis of predominant movement.

The code is available at the following GitHub Repository: [pose-estimation](#).